Marwadi University
Marwadi Chandarana Group

NAAC A+

Marwadi University

Faculty of Computer Applications

MCA Sem- II

Basics of Python
(05MC0202)

Unit – 2
Exception, Modules and File Handling

Exceptions

- Error in Python can be of two types i.e. Syntax errors and Exceptions. Errors are the problems in a program due to which the program will stop the execution.
- On the other hand, exceptions are raised when some internal events occur which changes the normal flow of the program.

Exceptions

- An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions.

- In general, when a Python script encounters a situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error.

Exceptions

- There are mainly three kinds of distinguishable errors in Python: syntax errors, exceptions and logical errors.
- Syntax errors are similar to grammar or spelling errors in a Language. If there is such an error in your code, Python cannot start to execute your code. You get a clear error message stating what is wrong and what needs to be fixed. Therefore, it is the easiest error type you can fix.
- Missing symbols (such as comma, bracket, colon), misspelling a keyword, having incorrect indentation are common syntax errors in Python.

Exceptions

- **Exceptions** may occur in syntactically correct code blocks at run time. When Python cannot execute the requested action, it terminates the code and raises an error message.
- Trying to read from a file which does not exist, performing operations with incompatible types of variables, dividing a number by zero are common **exceptions** that raise an error in Python.
- We must eliminate the **syntax errors** to run our Python code, while **exceptions** can be handled at runtime.

Exceptions

- Logical errors are the most difficult errors to fix as they don’t crash your code and you don’t get any error message.
- If you have logical errors, your code does not run as you expected.
- Using incorrect variable names, code that is not reflecting the algorithm logic properly, making mistakes on boolean operators will result in logical errors.

Exceptions

- **The difference between Syntax Error and Exceptions**
- Syntax Error: As the name suggest this error is caused by wrong syntax in the code. It leads to the termination of the program.

Exceptions

Example

```
# initialize the amount variable
amount = 10000
# check that You are eligible to purchase Dsa Self Paced or not
if(amount>2999)
    print("You are eligible to purchase Dsa Self Paced")
```

Output:

Invalid syntax

Exceptions

Exceptions: Exceptions are raised when the program is syntactically correct but the code resulted in an error. This error does not stop the execution of the program, however, it changes the normal flow of the program.

Exceptions

Example:

```
a=10
b=0
print(a/b)
print("hello")
```

Output:

```
Traceback (most recent call last):
  File "C:/Users/admin/Desktop/temp.py", line 4, in
<module>
    print(a/b)
ZeroDivisionError: division by zero
```

Exceptions

In the above example raised the ZeroDivisionError as we are trying to divide a number by 0.

Note: Exception is the base class for all the exceptions in Python.

Try and Except in Exception Handling

Let us try to handle division by zero error in python using Try and Except keywords :

```
# Python program to handle simple runtime error

a=10
b=0

try:
   print(a/b)
except Exception as e:
   print(e)
print("hello")
```

Output:
division by zero
hello

Try and Except in Exception Handling

Example :

```
a=10
b=0

try:
   print(a/b)
except Exception as e:
   print( "ohh its error",e)

finally:
   print("hello")
```

Output:

```
ohh its error division by zero
hello
```

Try and Except in Exception Handling

Else Clause

In python, you can also use else clause on the try-except block which must be present after all the except clauses. The code enters the else block only if the try clause does not raise an exception.

```
# Program to depict else clause with try-except
# Function which returns a/b
def fun(a , b):
    try:
        c = ((a+b)/(a-b))
    except ZeroDivisionError:
        print ("a/b result in 0")
    else:
        print (c)

# Driver program to test above function
fun(2.0, 3.0)
fun(3.0, 3.0)
```

The output for above program will be :

```
-5.0
a/b result in 0
```

Try and Except in Exception Handling

Finally Keyword in Python

Python provides a keyword finally, which is always executed after try and except blocks. The finally block always executes after normal termination of try block or after try block terminates due to some exception.

Syntax:

```
try:
    # Some Code....
except:
    # optional block
    # Handling of exception (if required)
else:
    # execute if no exception
finally:
   # Some code .....(always executed)
```

Try and Except in Exception Handling

Example:

```
a=10
b=0

try:
   print(a/b)
except Exception as e:
   print( "ohh its error",e)

finally:
   print("this is always executed")
```

Output:

ohh its error division by zero
this is always executed

Try and Except in Exception Handling

Multiple exception:

Example :

```
a=10
b=2
try:
   print(a/b)
   k=int(input("enter the number"))
   print(k)
except ZeroDivisionError as e:
   print("zero div error")
except ValueError as e:
   print("invalid input")
finally:
   print("this is always executed")
```

Output :

```
5.0
enter the number:p
invalid input
this is always executed
```

Try and Except in Exception Handling

```
try:
# code that may raise an exception
except Exception as e:
# code to handle the exception
```

```
try:
# code that may raise a ValueError exception
except ValueError as e:
# code to handle the ValueError exception
```

```
try:
# code that may raise a ValueError or TypeError exception
except (ValueError, TypeError) as e:
# code to handle the ValueError or TypeError exception
```

```
try:
# code that may raise any exception
except Exception as e:
# code to handle any exception
```

Try and Except in Exception Handling

OverflowError:

OverflowError in Python indicates that an arithmetic operation has exceeded the limits of the current Python runtime. This is typically due to excessively large float values, as integer values that are too big will opt to raise memory errors instead.

ImportError:

It is raised when you try to import a module which does not exist. This may happen if you made a typing mistake in the module name or the module doesn't exist in its standard path. In the example below, a module named "non_existing_module" is being imported but it doesn't exist, hence an import error exception is raised.

IndexError:

An IndexError exception is raised when you refer a sequence which is out of range. In the example below, the list abc contains only 3 entries, but the 4th index is being accessed, which will result an IndexError exception.

TypeError:

When two unrelated type of objects are combined, TypeErrorexception is raised.In example below, an int and a string is added, which will result in TypeError exception.

Try and Except in Exception Handling

IndentationError:

Unexpected indent. As mentioned in the "expected an indentedblock" section, Python not only insists on indentation, it insists on consistentindentation. You are free to choose the number of spaces of indentation to use, but you then need to stick with it.

Syntax errors:

These are the most basic type of error. They arise when the Python parser is unable to understand a line of code. Syntax errors are almost always fatal, i.e. there is almost never a way to successfully execute a piece of code containing syntax errors.

Run-time error:

A run-time error happens when Python understands what you are saying, but runs into trouble when following your instructions.

Key Error :

Python raises a KeyError whenever a dict() object is requested (using the format a = adict[key]) and the key is not in the dictionary.

Value Error:

In Python, a value is the information that is stored within a certain object. To encounter a ValueError in Python means that is a problem with the content of the object you tried to assign the value to.

Try and Except in Exception Handling

Different types of exceptions:

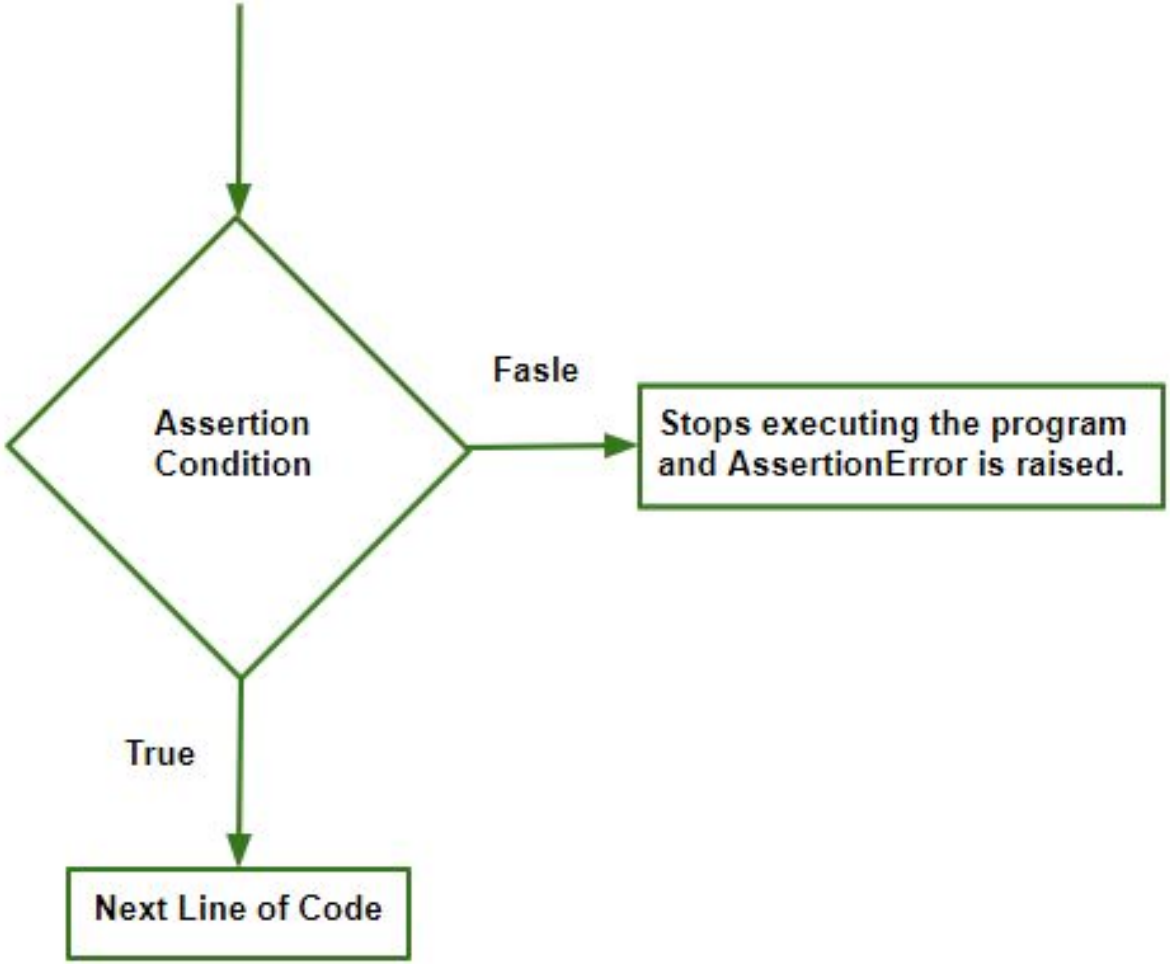
- ArrayIndexOutOfBoundException.
- ClassNotFoundException.
- FileNotFoundException.
- IOException.
- InterruptedException.
- NoSuchFieldException.
- NoSuchMethodException

Assertion Error

Assertion Error

- Assertion is a programming concept used while writing a code where the user declares a condition to be true using assert statement prior to running the module.
- If the condition is True, the control simply moves to the next line of code. In case if it is False the program stops running and returns AssertionError Exception.
- The function of assert statement is the same irrespective of the language in which it is implemented, it is a language-independent concept, only the syntax varies with the programming language.

Assertion Error



Assertion Error

Syntax of assertion:
assert condition, error_message(optional)

Example -1:

```
 # AssertionError with error_message.
x = 1
y = 0
assert y != 0, "Invalid Operation" # denominator can't be 0
print(x / y)
```

Output:

```
Traceback (most recent call last):
  File "/home/bafc2f900d9791144fbf59f477cd4059.py", line 4, in
    assert y!=0, "Invalid Operation" # denominator can't be 0
AssertionError: Invalid Operation
```

The default exception handler in python will print the error_message written by the programmer, or else will just handle the error without any message.
Both of the ways are valid.

Assertion Error

Handling AssertionError exception:
AssertionError is inherited from Exception class, when this exception occurs and raises AssertionError there are two ways to handle, either the user handles it or the default exception handler.
In Example 1 we have seen how the default exception handler does the work.
Now let's dig into handling it manually.

```
# Handling it manually
try:
    x = 1
    y = 0
    assert y != 0, "Invalid Operation"
    print(x / y)

# the errror_message provided by the user gets printed
except AssertionError as msg:
    print(msg)
```

Output :
Invalid Operation

Logging in Python

- Logging is a means of tracking events that happen when some software runs. Logging is important for software developing, debugging and running. If you don’t have any logging record and your program crashes, there are very little chances that you detect the cause of the problem
- Logging an exception in python with an error can be done in the logging.exception()method.
- This function logs a message with level ERROR on this logger.
- The arguments are interpreted as for debug().
- Exception info is added to the logging message. This method should only be called from an exception handler.

Logging in Python

Example :

```
import logging
try:
    printf("Hello")
except Exception as Argument:
    logging.exception("Error occured while printing Hello")
```

Output:

```
ERROR:root:Error occured while printing GeeksforGeeks
Traceback (most recent call last):
  File "/home/gfg.py", line 3, in
    printf("GeeksforGeeks")
NameError: name 'printf' is not defined
```

Logging in Python

Example -2 :

```
# importing the module
import logging

try:
   printf("Hello")
except Exception as Argument:

    # creating/opening a file
    f=open("/Users/vivekgondalia/Desktop/demofile2.txt", "w")

    # writing in the file
    f.write(str(Argument))

    # closing the file
    f.close()
```

Error message will be stored in file name *demofille2.txt* in same directory as code.

Module

- Consider a module to be the same as a code library.
- A file containing a set of functions you want to include in your application.

Example :

Save this code in a file named mymodule.py

```
def greeting(name):
  print("Hello, " + name)
```

Module

Use a Module

- Now we can use the module we just created, by using the import statement:

Example :

- Import the module named mymodule, and call the greeting function:

```
import mymodule
mymodule.greeting(“Kalpesh")
```

Module

- A module allows you to logically organize your Python code.
- Grouping related code into a module makes the code easier to understand and use.
- Simply, a module is a file consisting of Python code. A module can define functions, classes and variables. A module can also include runnable code.
- You can use any Python source file as a module by executing an import statement in some other Python source file.
- **Import modname**
- When the interpreter encounters an import statement, it imports the module if the module is present in the search path.
- Python's from statement lets you import specific attributes from a module into the current namespace.

Module

- **from modname import mod1,mod2**
- It is also possible to import all names from a module into the current namespace.
- from modname import *

Module

Example :

<u>Calc.py file :</u>

```
def add(a,b):
    return a+b

def sub(a,b):
    return a-b
```

<u>Main.py file code:</u>

```
import calc

a=10
b=20

ans=calc.add(a,b)

print(ans)
```

Module

OR----------

```
from calc import add,sub

a=10
b=20

ans=add(a,b)

print(ans)
```

OR----------

```
from calc import *
a=10
b=20

ans=add(a,b)

print(ans)
```

Re-naming a Module

Example :
Create an alias for mymodule called mx:

```
import mymodule as mx
a = mx.person1["age"]
print(a)
```

Module in detail

Built-in Modules in Python

One of the many superpowers of Python is that it comes with a “rich standard library”. This rich standard library contains lots of built-in modules. Hence, it provides a lot of reusable code.

To name a few, Python contains modules like “os”, “sys”, “datetime”, “random”.

You can import and use any of the built-in modules whenever you like in your program.

Importing built-in module

Let’s now import a built-in module into our Python shell:

```
>>> import random
```

To call the randint **function of** random, we simply need to use the dot notation:

```
>>> random.randint(20, 100)
```

The dir() function

Python dir() function

The dir() function will return the names of all the properties and methods present in a module.

```
>>> import random
>>> dir(random)
['BPF', 'LOG4', 'NV_MAGICCONST', 'RECIP_BPF', 'Random', 'SG_MAGICCONST',
'SystemRandom', 'TWOPI', '_Sequence', '_Set', '__all__', '__builtins__',
'__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__',
'__spec__', '_accumulate', '_acos', '_bisect', '_ceil', '_cos', '_e', '_exp', '_inst', '_log',
'_os', '_pi', '_random', '_repeat', '_sha512', '_sin', '_sqrt', '_test', '_test_generator',
'_urandom', '_warn', 'betavariate', 'choice', 'choices', 'expovariate', 'gammavariate',
'gauss', 'getrandbits', 'getstate', 'lognormvariate', 'normalvariate', 'paretovariate',
'randint', 'random', 'randrange', 'sample', 'seed', 'setstate', 'shuffle', 'triangular',
'uniform', 'vonmisesvariate', 'weibullvariate‘]
```

Difference between Modules and Libraries

Modules	Libraries
A module is a collection of code or functions that uses the .py extension.	A Python library is a set of related modules or packages bundled together.
It is used by the programmers as well as the developers.	It is mostly used by the community members, developers, and researchers.
Use of modules makes reading the code easier.	Libraries do not contribute in better readability.
Modules logically cluster the functionality that programmers can import to reuse their code or set of	Libraries make the collection of logically related code reusable for the programming language users, developers, and

Difference between Modules and Libraries

Whenever a programmer imports a module in a Python program, the interpreter scans for several locations to look for the module's definition or body.	We have to install the libraries in our Python project before using its modules or packages. Usually, we use the pip install command.
When a module is not found by Python's import statement, it searches for each directory within the shell variable i.e., PYTHONPATH. The PYTHONPATH is Python's environment variable consisting of a list of directories.	Whenever the Python interpreter do not found the Python library associated with the project, it shows an error message and the program ends abruptly.

Difference between Modules and Libraries

Modules are mostly written using valid Python statements or codes.	Libraries, mainly standard libraries, are mostly written using C language or Python.
The main focus of creating modules is to avoid DRY (Don’t Repeat Yourself).	Libraries do not have any such objective.
We can use the Python’s built-in dir() function to return a sorted list of strings holding the function names defined within a module.	There is not such explicit function that can return the number of modules a library contain. Still, programmers can use the help() to extract some information.
Example of popular built-in Python modules are os, sys, math, random, etc.	Example of popular built-in Python libraries are Pygame, Pytorch, matplotlib, etc.

The PYTHONPATH Variable

- The PYTHONPATH environment variable is a list of directories that Python will search for modules when importing them. It is a colon-separated list of directories, and can be set in the environment before starting Python, or in a Python script using the os.path.append() function.
- The PYTHONPATH variable is useful for a number of reasons. First, it allows you to import modules that are not installed in the default Python installation directory. Second, it allows you to organize your Python code more efficiently by grouping related modules together in a single directory. Third, it can be used to override the default search path for modules, which can be useful for debugging or testing purposes.
- To set the PYTHONPATH environment variable in the environment, you can use the following command:

The PYTHONPATH Variable

- export PYTHONPATH=/path/to/directory1:/path/to/directory2

- Pythonpath is a special environment variable that provides guidance to the Python interpreter about where to find various libraries and applications. It is similar to the PATH environment variable in other languages, such as C and Java, but has additional directories for Python modules.

Dates in Python

- A date in Python is not a data type of its own, but we can import a module named datetime to work with dates as date objects.

- Example

- Import the datetime module and display the current date:

```
import datetime
x = datetime.datetime.now()
print(x)
```

Dates in Python

```
import datetime
x = datetime.datetime.now()
print(x.year)
print(x.strftime("%A"))
```

You can also create a date object

```
import datetime
x = datetime.datetime(2020, 5, 17)
print(x)
```

The strftime() Method

- The datetime object has a method for formatting date objects into readable strings.

- The method is called strftime(), and takes one parameter, format, to specify the format of the returned string:

```
import datetime
x = datetime.datetime(2018, 6, 1)
print(x.strftime("%B"))
```

https://www.w3schools.com/python/python_datetime.asp

Math in Python

- Python has a set of built-in math functions, including an extensive math module, that allows you to perform mathematical tasks on numbers.

- min()
- max()
- abs()
- pow()

- Above are built-in functions. Python has Math module also for some mathematical functions

- https://www.w3schools.com/python/module_math.asp

File Handling in Python

- File handling is an important part of any web application.
- Python has several functions for creating, reading, updating, and deleting files.
- The key function for working with files in Python is the open() function.
- The open() function takes two parameters; filename, and mode.
- There are four different methods (modes) for opening a file:

File Handling in Python

- "r" - Read - Default value. Opens a file for reading, error if the file does not exist
- "a" - Append - Opens a file for appending, creates the file if it does not exist
- "w" - Write - Opens a file for writing, creates the file if it does not exist
- "x" - Create - Creates the specified file, returns an error if the file exists

File Handling in Python

- In addition you can specify if the file should be handled as binary or text mode
- "t" - Text - Default value. Text mode
- "b" - Binary - Binary mode (e.g. images)

File Handling in Python

- To open a file for reading it is enough to specify the name of the file:

```
f = open("demofile.txt")
```

- The code above is the same as:

```
f = open("demofile.txt", "rt")
```

- Because "r" for read, and "t" for text are the default values, you do not need to specify them.

- Note: Make sure the file exists, or else you will get an error.

Reading files in python

After we open a file, we use the read() method to read its contents. For example,

```
# open a file
 file1 = open("test.txt", "r")
# read the file
 read_content = file1.read()
print(read_content)
```

Closing file in python

When we are done with performing operations on the file, we need to properly close the file.
Closing a file will free up the resources that were tied with the file. It is done using the close() method in Python. For example,

```
# open a file
file1 = open("test.txt", "r")
# read the file
read_content = file1.read() print(read_content)
 # close the file
file1.close()
```

Use of with…open

In Python, we can use the with...open syntax to automatically close the file. For example,

```
with open("test.txt", "r") as file1:
    read_content = file1.read()
    print(read_content)
```

Note: Since we don't have to worry about closing the file, make a habit of using
the with...open syntax.

Writing file in python

There are two things we need to remember while writing to a file.

- If we try to open a file that doesn't exist, a new file is created.
- If a file already exists, its content is erased, and new content is added to the file.

Writing file in python

In order to write into a file in Python, we need to open it in write mode by passing "w" inside open() as a second argument.

Suppose, we don't have a file named **test2.txt**. Let's see what happens if we write contents to the test2.txt file.

```
with open(test2.txt', 'w') as file2:
 # write contents to the test2.txt file
    file2.write('Programming is Fun.')
    file2.write('Python for beginners')
```

Here, a new test2.txt file is created and this file will have contents specified inside the write() method.

Writing file in python

When you access a file on an operating system, a file path is required. The file path is a string that represents the location of a file. It's broken up into three major parts:

1. **Folder Path:** the file folder location on the file system where subsequent folders are separated by a forward slash / (Unix) or backslash \ (Windows)
2. **File Name:** the actual name of the file
3. **Extension:** the end of the file path pre-pended with a period (.) used to indicate the file type

Writing file in python

A path is either *relative* or *absolute*. An absolute path always contains the root element and the complete directory list required to locate the file. For example, /home/sally/statusReport is an absolute path. All of the information needed to locate the file is contained in the path string.

A relative path needs to be combined with another path in order to access a file. For example, joe/foo is a relative path. Without more information, a program cannot reliably locate the joe/foo directory in the file system.

Writing file in python

To find an absolute path in Python you import the os module then you can find the current working directory using **os.path.abspath("insert-file-name-here")** in your Python script.

to find the absolute path in Python

To get the absolute path in Python you write the following code:

import os os.path.abspath("src/examplefile.txt")

Writing file in python

to find the relative path in Python

To get the relative path in Python you write the following code:

```
import os
absolute_path = os.path.dirname(__file__)
relative_path = "src/lib“
 full_path = os.path.join(absolute_path, relative_path)
```

Writing file in python

First, you have to import the os module in Python so you can run operating system functionalities in your code.
Then you create the variable absolute_path which fetches the current directory relative to the root folder. This is the full path to your working directory, in this case, ~/home/projects/example-project/.

The advantage to getting the absolute path on your operating system is that it makes it possible to run the script on different systems on different working directories.

Writing file in python

The relative_path variable is a string in which you define the location of the folder that you want to fetch relative to the working directory.
In this case, "src/lib".

Then we use the absolute_path and combine it via join with the relative_path to get the full path to the lib folder which results in:

/Users/dannysteenman/projects/example-project/src/lib/

Writing file in python

Manipulating Pathnames

Here are some different examples used mentioned below −

- Getting the main filename from the file path
- Getting the directory name from the file path
- Joining path components together
- Expanding the User's Home Directory
- Splitting the file extension from the file path

Writing file in python

Getting the main filename from the file path

The following program returns the main file name from the input file using the os.path.basename() function −

```
# importing os module
import os
# input path of the file
inputFilepath = 'C:/Users/cirus/Desktop/test.pdf'
# Printing the given input path
print("Give Input Path is:",inputFilepath)
# getting the last component(main file name )of the input file path print("Base Name of the given path is :", os.path.basename(inputFilepath))
```

Writing file in python

Getting the directory name from the file path

Use the **os.path.dirname()** function(returns the directory name from the given file path) to get the directory/folder of the given input file path by passing the input file path as an argument to it.

Writing file in python

Example

The following program returns the directory name from the input file path using the os.path.dirname() function −

```
import os
inputFilepath = 'C:/Users/cirus/Desktop/test.pdf'
print("Give Input Path is:",inputFilepath)
# getting the directory/folder path from the input file path using dirname() function
print("Directory path of the given path is: ", os.path.dirname(inputFilepath))
```

Writing file in python

Joining path components together

os.path.join() function

Python's os.path.join() function effectively joins one or more path components. With the exception of the last path component, this approach concatenates different path components by placing exactly one directory separator **('/')** after each non-empty portion. A directory separator ('/') is added at the end of the last path component to be joined is empty.

All previously joined components are deleted if a path component represents an absolute path, and joining moves on from the absolute path component.

Writing file in python

Example

The following program joins the given path components with the base name using the os.path.join() function −

```
# importing os module
import os
 # input path of the file
inputFilepath = 'C:/Users/cirus/Desktop/kohli.pdf‘
 # Printing the given input path
 print("Give Input Path is:",inputFilepath)
# joining the components to the main file name of the input file path
print("Joining the given paths to input Path:\n", os.path.join('tutorials',
'python', os.path.basename(inputFilepath)))
```

Output

Give Input Path is: C:/Users/cirus/Desktop/kohli.pdf

Joining the given paths to input Path: tutorials/python/kohli.pdf

Writing file in python

Splitting the file extension from file path:

os.path.splitext() function − Splits the file path name into a pair root and extension. The root here is everything except the file extension.

If the given file path has no extensions, then the extension is empty. The given path will be ignored if it has a leading period ('.').

Syntax

Below is the syntax of the function.

os.path.splitext(path)

Use the **os.path.splitext()** function to split the file path and file extension from the input file path.

Writing file in python

Example

The following program splits the main file path and file extension from the input file path using the os.path.splitext() function −

```
import os
# input path of the file
 inputFilepath ='C:/Users/cirus/Desktop/tutorial.pdf'
# Printing the given input path
 print("Give Input Path is:",inputFilepath)
# splitting the file path and file extension from the input file path
 # using the splitext() function
print("Splitting the given path by extension :\n" os.path.splitext(inputFilepath))
```

Writing file in python

Output

On execution, the above program will generate the following

output –

Give Input Path is: C:/Users/cirus/Desktop/tutorial.pdf
Splitting the given path by extension:
('C:/Users/cirus/Desktop/tutorial, '.pdf')

OS Path Module

Following are some functions of OS Path module.

1. os.path.basename(path) : It is used to return the basename of the file . This function basically return the file name from the path given.

```
import os
out = os.path.basename("/baz/foo")
print(out)
```

OS Path Module

2. os.path.dirname(path) : It is used to return the directory name from the path given. This function returns the name from the path except the path name.

```
import os
out = os.path.dirname("/baz/foo")
print(out)
```

OS Path Module

3. os.path.isabs(path) : It specifies whether the path is absolute or not. In Unix system absolute path means path begins with the slash(‘/’) and in Windows that it begins with a (back)slash after chopping off a potential drive letter.

```
import os
out = os.path.isabs("/baz/foo")
print(out)
```

OS Path Module

4. os.path.isdir(path) : This function specifies whether the path is existing directory or not.

```
import os
out = os.path.isdir("C:\\Users")
print(out)
```

OS Path Module

5. os.path.isfile(path) : This function specifies whether the path is existing file or not.

```
import os
out = os.path.isfile("C:\\Users\foo.csv")
print(out)
```

Getting file information in python

Because Python is a server-side language, it can access and manipulate the file system just like any other language.
That means that you can use Python to get information about files, such as their size, creation date, and more.

Get the file size

Now that you've imported the os module, you can use the os.path.getsize() function to get the size of a file.
First define a file path to the file you want to get the size of, and keep in mind that the path will be different depending on your operating system.

Getting file information in python

file_path = "C:\\Users\\sabe\\text.txt" # Windows

Then use the os.path.getsize() function to get the size of the file:

```
import os
file_path = "/Users/sabe/text.txt“
 file_size = os.path.getsize(file_path) print(file_size)
```

Getting file information in python

Get the file creation date

To get the creation date of a file, you can use the os.path.getctime() function.

```
import os
file_path = "/Users/sabe/text.txt"
file_creation_date = os.path.getctime(file_path)
print(file_creation_date)
```

This returns the number of seconds since the epoch, which is January 1, 1970.

Getting file information in python

Get the file modification date

To get the modification date of a file, you can use the os.path.getmtime() function.

```
import os file_path = "/Users/sabe/text.txt" file_modification_date = os.path.getmtime(file_path) print(file_modification_date)
```

Getting file information in python

How to get the stats of a file

Another thing you can fetch about a file is its stats.

The os.stat() function returns a stat_result object that contains a number of attributes about the file.

```
import os
file_path = "/Users/sabe/text.txt"
file_stats = os.stat(file_path)
print(file_stats)
```

These attributes include the file size, file owner id, file group id, permissions, and more.

Getting file information in python

Traversing Directories and Processing Files

A common programming task is walking a directory tree and processing files in the tree. Let's explore how the built-in Python function os.walk() can be used to do this. os.walk() is used to generate filename in a directory tree by walking the tree either top-down or bottom-up.

The following is an example that shows you how to list all files and directories in a directory tree using os.walk().
os.walk() defaults to traversing directories in a top-down manner:

Getting file information in python

```
# Walking a directory tree and printing the names of the directories and files
for dirpath, dirnames, files in os.walk('.'):
    print(f'Found directory: {dirpath}')
    for file_name in files:
        print(file_name)
```

os.walk() returns three values on each iteration of the loop:

1. The name of the current folder
2. A list of folders in the current folder
3. A list of files in the current folder

Getting file information in python

To traverse the directory tree in a bottom-up manner, pass in a topdown=False keyword argument to os.walk():

```
for dirpath, dirnames, files in os.walk('.', topdown=False):
    print(f'Found directory: {dirpath}')
    for file_name in files:
        print(file_name)
```

Deleting file in python

Deleting Files in Python

To delete a single file, use pathlib.Path.unlink(), os.remove(). or os.unlink(). os.remove() and os.unlink() are semantically identical. To delete a file using os.remove(), do the following:

```
import os
data_file = 'C:\\Users\\vuyisile\\Desktop\\Test\\data.txt'
os.remove(data_file)
```

Examples

Example:

```
# Open function to open the file "MyFile1.txt"
# (same directory) in append mode and
file1 = open("MyFile1.txt","a")

# store its reference in the variable file1
# and "MyFile2.txt" in D:\Text in file2
file2 = open(r"D:\Text\MyFile2.txt","w+")
```

Example :

```
# Opening and Closing a file "MyFile.txt"
# for object name file1.
file1 = open("MyFile.txt","a")
file1.close()
```

Examples

```
# Program to show various ways to read and
# write data in a file.
file1 = open("myfile.txt","w")
L = ["This is Delhi \n","This is Paris \n","This is London \n"]

# \n is placed to indicate EOL (End of Line)
file1.write("Hello \n")
file1.writelines(L)
file1.close() #to change file access modes

file1 = open("myfile.txt","r+")

print("Output of Read function is ")
print(file1.read())
print()
```

Examples

```
# seek(n) takes the file handle to the nth
# bite from the beginning.
file1.seek(0)

print( "Output of Readline function is ")
print(file1.readline())
print()

file1.seek(0)
```

Examples

```
# To show difference between read and readline
print("Output of Read(9) function is ")
print(file1.read(9))
print()
file1.seek(0)

print("Output of Readline(9) function is ")
print(file1.readline(9))

file1.seek(0)
# readlines function
print("Output of Readlines function is ")
print(file1.readlines())
print()
file1.close()
```

Examples

Appending to a file

```
# Python program to illustrate
# Append vs write mode
file1 = open("myfile.txt","w")
L = ["This is Delhi \n","This is Paris \n","This is London \n"]
file1.writelines(L)
file1.close()

# Append-adds at last
file1 = open("myfile.txt","a")#append mode
file1.write("Today \n")
file1.close()
```

Examples

```
file1 = open("myfile.txt","r")
print("Output of Readlines after appending")
print(file1.readlines())
print()
file1.close()

# Write-Overwrites
file1 = open("myfile.txt","w")#write mode
file1.write("Tomorrow \n")
file1.close()
```

Examples

```
file1 = open("myfile.txt","r")
print("Output of Readlines after writing")
print(file1.readlines())
print()
file1.close()
```

File Object

What is the File Object?

- Python **file object provides methods and attributes to access and manipulate files**. Using file objects, we can read or write any files.
- Whenever we open a file to perform any operations on it, Python returns a file object. To create a file object in Python use the built-in functions, such as open() and os.popen().
- IOError exception is raised when a file object is misused, or file operation fails for an I/O-related reason. For example, when you try to write to a file when a file is opened in read-only mode.

Text Files

Text files (TextIOWrapper)

The text file type is most common. Usually, we use text files to store character data or storing information in plain text with no special formatting beyond basic fonts and font styles.

We open a text file using the open() function. For example, open('test'.txt', 'r'). When we open a text file, it returns a TextIOWrapper file object.

Example

```
file = open('test.txt', 'w')
print(type(file))
# Output: <class '_io.TextIOWrapper'>
```

Binary Files

Binary Files (BufferedReader and BufferedWriter)

- Data is stored on a disk in the form of binary. For example, we use binary files to store data like images or videos. Binary files are a computer-readable form of storing data.
- A program is needed to interpret the data in a binary file and display it to the user. The binary files are also called buffered files. This file type is used for reading and writing binary data.
- Open the binary files using the open() function in binary mode.

Binary Files

For example, open('abc.txt', 'rb'). It opens the file to read-only in binary mode. The file pointer exists at the beginning of the file.

The open() function will return the BufferedReader when we open the binary file for reading and the BufferedWriter file object when we open a binary file for writing.

Example

```
file = open('test.txt', 'rb') print(type(file))
# Output: <class '_io.BufferedReader'>
```

Writing to file

Writing to a file

There are two ways to write in a file.

1. **write() :** Inserts the string str1 in a single line in the text file.

   File_object.write(str1)

   2. writelines() : For a list of string elements, each string is inserted in the text file.Used to insert multiple strings at a single time.

   File_object.writelines(L) for L = [str1, str2, str3]

Reading from file

Reading from a file

There are three ways to read data from a text file.

1. **read() :** Returns the read bytes in form of a string. Reads n bytes, if no n specified, reads the entire file.
File_object.read([n])

2. **readline() :** Reads a line of the file and returns in form of a string. For specified n, reads at most n bytes. However, does not reads more than one line, even if n exceeds the length of the line.
File_object.readline([n])

Reading from file

3. readlines() : Reads all the lines and return them as each line a string element in a list.

File_object.readlines()

Note: ‘\n’ is treated as a special character of two bytes

Examples

```
# Program to show various ways to read and
# write data in a file.
file1 = open("myfile.txt","w")
L = ["This is Delhi \n","This is Paris \n","This is London
\n"]

# \n is placed to indicate EOL (End of Line)
file1.write("Hello \n")
file1.writelines(L)
file1.close()
#to change file access modes
```

Examples

```
file1 = open("myfile.txt","r+")

print("Output of Read function is ")
print(file1.read())
print()

# seek(n) takes the file handle to the nth
# bite from the beginning.
file1.seek(0)

print( "Output of Readline function is ")
print(file1.readline())
print()
file1.seek(0)
```

Examples

```
# To show difference between read and readline
print("Output of Read(9) function is ")
print(file1.read(9))
print()

file1.seek(0)

print("Output of Readline(9) function is ")
print(file1.readline(9))
```

Examples

```
file1.seek(0)
# readlines function
print("Output of Readlines function is ")
print(file1.readlines())
print()
file1.close()
```

```
Output of Read function is
Hello
This is Delhi
 This is Paris
This is London
Output of Readline function is
 Hello
```

```
Output of Read(9) function is
Hello
Th Output of Readline(9) function is
 Hello
Output of Readlines function is
['Hello \n', 'This is Delhi \n', 'This is Paris
\n', 'This is London \n']
```

Working with Binary Files

- **Working with Binary Files in Python**
- Python has tools for working with binary files. Binary files use strings of type bytes. This means when reading binary data from a file, an object of type bytes is returned.
- The binary file is opened using the open() function, whose mode parameter contains the character **'b'**. More details about opening/closing binary files are described **here**.
- Unlike text files, binary files do not convert characters to the end of the string '\n'.
- An example demonstrating the features of the presentation of information in binary files.

Working with Binary Files

```
# Open binary file for reading
f = open('myfile1.bin', 'rb')
# Get a string from binary file
d = f.read()
# Display this string.
 # The output will be as a string of characters
 print("d = ", d)
# d = b'\x80\x03]q\x00(K\x01\x88G@\x07\n=p\xa3\xd7\ne.‘
 # If print as a separate character, # then the character code will be
displayed - as an integer
print("d[5] = ", d[5])  # d[5] = 40
print("d[0] = ", d[0]) # d[0] = 128
 # Use bin function for single character
print(bin(d[2])) # 0b1011101 f.close()
```

Working with Binary Files

Output:

d = b'\x80\x03]q\x00(K\x01\x88G@\x07\n=p\xa3\xd7\ne.' d[5] = 40
d[0] = 128
0b1011101

Based on the example above, the following conclusions can be drawn:

✔ a string of binary data is output as a string;

✔ a single character (element) of binary data is represented as 8-bit integers.

Zipping and Unzipping files in Python

To zip a file, you can use the zipfile module. Here is an example:

```
import zipfile
# Create a ZipFile object
with zipfile.ZipFile('my_zip_file.zip', 'w') as zip_file:
    # Add a file to the zip file
    zip_file.write('my_file.txt')
```

Zipping and Unzipping files in Python

To unzip a file, you can also use the zipfile module. Here is an example:

```
import zipfile

# Create a ZipFile object
with zipfile.ZipFile('my_zip_file.zip', 'r') as zip_file:
    # Extract all the files from the zip file
    zip_file.extractall()
```

Zipping and Unzipping files in Python

You can also use the shutil module to zip and unzip files. Here is an example of how to zip a file using the shutil module:

```
import shutil

# Zip a file
shutil.make_archive('my_zip_file', 'zip', 'my_file.txt')
```

Zipping and Unzipping files in Python

Here is an example of how to unzip a file using the shutil module:

```
import shutil

# Unzip a file
shutil.unpack_archive('my_zip_file.zip', 'my_directory')
```

Marwadi University
Marwadi Chandarana Group

NAAC A+

Thank You